

Tabelas Hash

COMP0497 – Algoritmos e Estrutura de Dados 1

Prof. Dr. André Yoshiaki Kashiwabara

Motivação: por que precisamos de hash?

Tabelas de Endereçamento Direto

Tabelas Hash com Encadeamento

Funções Hash

Hashing de Strings

Endereçamento Aberto

Hashing Perfeito

Síntese

**Motivação: por que precisamos
de hash?**

- **Python:** todo `dict` e `set` é uma tabela hash (CPython usa endereçamento aberto com sondagem perturbada).
- **Java:** `HashMap`, `HashSet`, `ConcurrentHashMap` (encadeamento com conversão para árvore quando uma lista cresce demais).
- **Bancos de dados:** índices hash, `GROUP BY`, joins por hash.
- **Sistemas:** caches (Redis, Memcached), tabelas de roteamento, deduplicação.
- **Compiladores:** tabela de símbolos (variáveis $\rightarrow$ tipo, escopo, endereço).
- **Bioinformática:** indexar k -mers de DNA, busca em genomas.
- **Versionamento (Git):** cada objeto é endereçado pelo hash de seu conteúdo.

- Queremos uma estrutura que suporte três operações sobre um conjunto dinâmico de chaves:
 - INSERT(T, x)
 - SEARCH(T, k)
 - DELETE(T, x)
- **Ideal:** cada uma em $O(1)$ *esperado*.

Comparação rápida

Estrutura	Busca	Inserção	Remoção
Lista ligada	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$
Array ordenado	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$
Árvore balanceada (AVL, R-B)	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$
Tabela hash	$\Theta(1)$ esperado	$\Theta(1)$ esperado	$\Theta(1)$ esperado

Cenário: contar palavras distintas em um corpus de $n = 10^7$ palavras.

Estrutura	Operações ($\sim n \log n$ ou n)	Tempo estimado
Lista ligada (busca linear)	$\sim 5 \times 10^{13}$	dias
Árvore balanceada	$\sim 2,3 \times 10^8$	~ 5 s
Tabela hash	$\sim 10^7$	$\sim 0,1$ s

- Hash não é mais rápido por mágica: ele *evita comparar chaves* ao calcular o endereço diretamente.
- Preço: precisamos pensar em **colisões** (duas chaves caem no mesmo slot).
- Garantia é em *expectativa*, não em pior caso (exceto hashing perfeito).

- Um compilador precisa associar nomes de variáveis (contador, idade, x) a informações (tipo, escopo, endereço).
- São poucos identificadores ativos por vez (digamos, 10^4), porém o *universo* possível é gigantesco: todas as strings ASCII de até 30 caracteres $\Rightarrow \sim 128^{30}$.
- Não dá para reservar memória para cada string possível.
- **Solução:** converter cada chave em um índice de um array pequeno via uma *função hash*.

Intuição

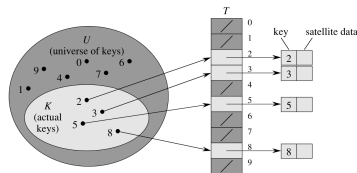
A função hash funciona como o cadastro do CPF em uma fila do banco: o número do CPF tem 11 dígitos, mas o sistema usa apenas os *últimos dois* para distribuir clientes em 100 guichês.

Tabelas de Endereçamento Direto

- Suponha que o universo $U = \{0, 1, \dots, m - 1\}$ é **pequeno**.
- Usamos um array $T[0 .. m - 1]$ tal que

$$T[k] = \begin{cases} x, & \text{se } x.\text{key} = k \\ \text{NIL}, & \text{caso contrário} \end{cases}$$

- Todas as operações são $\Theta(1)$ *garantido*.



Endereçamento direto.

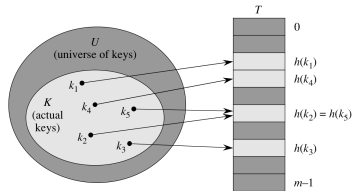
DIRECT-ADDRESS-SEARCH(T, k)	DIRECT-ADDRESS-INSERT(T, x)	DIRECT-ADDRESS-DELETE(T, k)
1 return $T[k]$	1 $T[x.key] = x$	1 $T[x.key] = \text{NIL}$

Quando *não* funciona

Se $|U|$ é enorme (ex.: chaves de 64 bits $\Rightarrow 2^{64}$ posições), o array é inviável. E se $|K| \ll |U|$ (poucas chaves ativas), o espaço é quase todo desperdiçado.

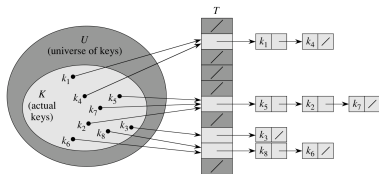
Tabelas Hash com Encadeamento

- **Ideia:** use um array $T[0..m-1]$ com $m \ll |U|$.
- Uma **função hash**
 $h: U \rightarrow \{0, 1, \dots, m-1\}$ comprime chaves em índices.
- Dizemos que “ k hash-eia no slot $h(k)$ ”.
- Inevitavelmente, $h(k_2) = h(k_5)$ para chaves distintas $\Rightarrow$ **colisão**.



Função hash h mapeia U em m slots.

- Cada slot $T[j]$ guarda o *cabeça* de uma lista ligada.
- Todas as chaves que hash-eiam em j vão para a lista $T[j]$.
- Operações:
 - Inserção: $O(1)$ (insere no início da lista).
 - Busca: proporcional ao comprimento da lista.
 - Remoção: $O(1)$ com lista duplamente ligada.



Encadeamento: cada slot aponta para uma lista.

Dados: $m = 7$, $h(k) = k \bmod 7$. Inserir 50, 700, 76, 85, 92, 73, 101.

k	$k \bmod 7$	vai para o slot
50	$50 = 7 \cdot 7 + 1$	1
700	$700 = 7 \cdot 100 + 0$	0
76	$76 = 7 \cdot 10 + 6$	6
85	$85 = 7 \cdot 12 + 1$	1 (colisão com 50)
92	$92 = 7 \cdot 13 + 1$	1 (colisão com 50, 85)
73	$73 = 7 \cdot 10 + 3$	3
101	$101 = 7 \cdot 14 + 3$	3 (colisão com 73)

Resultado: listas $T[0] = [700]$, $T[1] = [92, 85, 50]$, $T[3] = [101, 73]$, $T[6] = [76]$.

Comprimento médio $= 7/7 = 1 \Rightarrow \alpha = 1$. Buscas custam em média $\Theta(2)$.

Definição: fator de carga α

Para uma tabela T com m slots armazenando n chaves, definimos

$$\alpha = \frac{n}{m}.$$

α é o *comprimento médio* de uma lista no encadeamento.

Hipótese de hashing uniforme simples: cada chave tem probabilidade $1/m$ de cair em qualquer slot, independentemente das outras.

Teorema (11.1–11.2 do Cormen): sob hashing uniforme simples, uma busca (com ou sem sucesso) leva em média

$$\Theta(1 + \alpha).$$

Se $n = O(m)$, então $\alpha = O(1)$ e a busca é $\Theta(1)$ em média.

Funções Hash

- **Determinística:** a mesma chave sempre vai ao mesmo slot.
- **Computável rapidamente:** idealmente $O(1)$.
- **Distribuição uniforme:** aproxima a hipótese de hashing uniforme.
- **Independente de padrões nas chaves:** chaves parecidas (x_1, x_2, x_3) devem ir a slots diferentes.

Não confunda

Função hash $\neq$ função hash *criptográfica* (SHA-256, etc.). Para tabelas hash queremos velocidade e boa distribuição estatística, não resistência a adversários.

Fórmula

$$h(k) = k \bmod m$$

- Simples e rápido (um único mod).
- Escolha de m é crítica:
 - Evitar $m = 2^p$: $h(k)$ pega só os p bits menos significativos de k , descartando o resto.
 - **Boa prática:** m primo, distante de potências de 2 e de 10.
- Exemplo: para $n = 2000$ chaves com $\alpha \approx 3$, escolha $m = 701$ (primo perto de $2000/3$).

Por que $m = 2^p$ é ruim?

Exemplo: $m = 8 = 2^3$, $h(k) = k \bmod 8$.

Considere chaves cujos 3 bits menos significativos são todos iguais:

k	binário	$k \bmod 8$	slot
8	0000 <u>1000</u>	0	0
16	0001 <u>0000</u>	0	0
24	0001 <u>1000</u>	0	0
32	0010 <u>0000</u>	0	0

Todas no mesmo slot. A função descartou $|k|/8$ bits de informação.

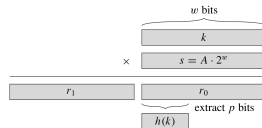
Com $m = 7$ (primo):

k	$k \bmod 7$	slot
8	1	1
16	2	2
24	3	3
32	4	4

Fórmula

$$h(k) = \lfloor m(kA \bmod 1) \rfloor, \quad 0 < A < 1.$$

- Vantagem: o valor de m *não é crítico* (qualquer potência de 2 serve).
- Knuth sugere
 $A \approx (\sqrt{5} - 1)/2 \approx 0,6180\dots$ (razão áurea).
- Implementação eficiente em palavras de w bits: multiplicar e extrair p bits.



Como ler a figura: a chave k ocupa uma palavra de w bits. Multiplica-se $k \cdot s$, com $s = \lfloor A \cdot 2^w \rfloor$, gerando um produto de $2w$ bits dividido em duas metades: a parte alta r_1 (descartada) e a parte baixa r_0 , que representa $kA \bmod 1$ em ponto fixo. Os p bits mais significativos de r_0 são tomados como $h(k)$, equivalente a $\lfloor m \cdot (kA \bmod 1) \rfloor$ quando $m = 2^p$. Tudo se reduz a uma multiplicação e um *shift*.

Dados: $k = 123\,456$, $m = 10\,000$, $A = (\sqrt{5} - 1)/2 \approx 0,6180339887$.

Passo 1. Multiplique:

$$kA = 123\,456 \cdot 0,6180339887 \approx 76\,300,004115\dots$$

Passo 2. Tome a parte fracionária:

$$kA \bmod 1 \approx 0,004115\dots$$

Passo 3. Multiplique por m e arredonde para baixo:

$$h(k) = \lfloor 10\,000 \cdot 0,004115\dots \rfloor = \lfloor 41,15\dots \rfloor = 41.$$

Compare com a divisão: $k \bmod 10\,000 = 3\,456$. Resultados *muito* diferentes.

Por que $A = (\sqrt{5} - 1)/2$?

A razão áurea minimiza aglomerações de $\{kA \bmod 1\}_{k=1,2,\dots}$ no intervalo $[0, 1)$ (teorema de Vera Sós sobre as três distâncias). Knuth: “works reasonably well” para qualquer chave.

- **Problema:** para qualquer função hash fixa, existe um conjunto de chaves que colide *todas* no mesmo slot.
- Um adversário malicioso pode forçar $\Theta(n)$ por operação.
- **Ideia:** escolher h *aleatoriamente* de uma família $\mathcal{H}$ no início da execução.

Família universal $\mathcal{H}$

$\mathcal{H}$ é *universal* se para cada par $k \neq \ell$,

$$\Pr_{h \in \mathcal{H}} [h(k) = h(\ell)] \leq \frac{1}{m}.$$

Resultado: com $\mathcal{H}$ universal, a expectativa de busca cai para $\Theta(1 + \alpha)$ *independentemente das chaves* escolhidas pelo adversário.

Construção

Escolha um primo $p > \max(K)$. Para cada par (a, b) com $a \in \{1, \dots, p-1\}$ e $b \in \{0, \dots, p-1\}$, defina

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m.$$

A família $\mathcal{H} = \{h_{a,b}\}$ tem $p(p-1)$ funções e é *universal*.

Algoritmo: ao iniciar a tabela, sorteie a e b uma única vez.

Exemplo: $p = 17$, $m = 6$, $a = 3$, $b = 4$.

k	$ak + b$	$\bmod 17$	$\bmod 6$	slot
2	$3 \cdot 2 + 4 = 10$	10	4	4
5	$3 \cdot 5 + 4 = 19$	2	2	2
8	$3 \cdot 8 + 4 = 28$	11	5	5
11	$3 \cdot 11 + 4 = 37$	3	3	3
14	$3 \cdot 14 + 4 = 46$	12	0	0

Note que $28 = 1 \cdot 17 + 11$, $37 = 2 \cdot 17 + 3$, $46 = 2 \cdot 17 + 12$. Sem aglomeração.

Observação: ataques reais (*hash-flooding DoS*, 2003 e 2011) levaram Python, Ruby e Java a adotar hashes randomizados por padrão.

Hashing de Strings

- Em quase toda aplicação real, as chaves são *strings*: nomes, URLs, identificadores, sequências de DNA.
- Uma string $s = s_0 s_1 \dots s_{L-1}$ é uma sequência de bytes (ou *code units*). Como transformá-la num inteiro?
- **Tentação ingênua:** somar os códigos dos caracteres.

$$h(s) = \left(\sum_{i=0}^{L-1} s_i \right) \bmod m.$$

- **Problema:** "abc", "bca" e "cab" colidem (mesmos caracteres). Anagramas viram um campo minado.

Critério-chave

Bom hash de strings tem que ser sensível à *posição* de cada caractere, não só ao conjunto.

Veja a string como os coeficientes de um polinômio em uma base b .

$$h(s) = (s_0 b^{L-1} + s_1 b^{L-2} + \dots + s_{L-1} b^0) \bmod p.$$

- b é uma base (tipicamente 31, 33, 37, 131 ou um primo).
- p é um primo grande ($\sim 10^9$) ou $2^{32} / 2^{64}$ (overflow natural).
- Posições diferentes $\Rightarrow$ potências diferentes de $b \Rightarrow$ anagramas não colidem.

Avaliação eficiente (regra de Horner)

$$h(s) = (\dots((s_0 \cdot b + s_1) \cdot b + s_2) \cdot b + \dots + s_{L-1}) \bmod p.$$

Custo: L multiplicações e L somas $\Rightarrow \Theta(L)$.

Java usa exatamente o hash polinomial com $b = 31$ e overflow em 32 bits:

```
public int hashCode() {  
    int h = 0;  
    for (int i = 0; i < length(); i++)  
        h = 31 * h + charAt(i);  
    return h;  
}
```

- Por que 31? Primo ímpar; $31 \cdot h = (h \ll 5) - h$ (otimização clássica).
- **Cuidado:** hashCode() *não é* o índice da tabela. HashMap aplica $(h \oplus (h \gg 16)) \& (m-1)$ para misturar os bits altos.
- **Python** (CPython) usa um polinômio com base 1000003 e *semente randomizada* a cada execução (defesa contra hash-flooding).

Exemplo numérico: $h(\text{"cat"})$ em Java



Caracteres: 'c' = 99, 'a' = 97, 't' = 116. Base $b = 31$.

Aplicando Horner:

$$h_0 = 0$$

$$h_1 = 31 \cdot 0 + 99 = 99$$

$$h_2 = 31 \cdot 99 + 97 = 3069 + 97 = 3166$$

$$h_3 = 31 \cdot 3166 + 116 = 98146 + 116 = 98262$$

Então `"cat".hashCode() = 98262` (confere com a JVM).

Para uma tabela com $m = 16$ slots: índice = $98262 \bmod 16 = 6$.

Verificação: $98262 = 16 \cdot 6141 + 6 \Rightarrow$ resto 6.

Comparação: "act" $\rightarrow 31 \cdot (31 \cdot 97 + 99) + 116 = 31 \cdot 3106 + 116 = 96402$.

Slots diferentes. A ingênua soma daria $99 + 97 + 116 = 312$ para ambos.

Problema: encontrar um padrão P (tamanho L) num texto T (tamanho n).

Ideia ingênua: hashear cada subjanela de $T \Rightarrow \Theta(nL)$.

Hash rolante: se sabemos $h(T[i .. i+L-1])$, conseguimos $h(T[i+1 .. i+L])$ em $O(1)$:

Recorrência

$$h_{i+1} = (b \cdot (h_i - T_i \cdot b^{L-1}) + T_{i+L}) \bmod p.$$

- Tirar o caractere mais antigo (multiplicado por b^{L-1}) e empurrar o novo.
- Custo total da busca: $\Theta(n + L)$ esperado.
- Em caso de *match* de hash, comparamos as strings byte-a-byte para confirmar (defesa contra falso positivo).

Padrão: $P = \text{"AB"}$ **Texto:** $T = \text{"ABXAB"}$

Base $b = 256$, primo $p = 101$. Códigos: $A=65$, $B=66$, $X=88$.

Hash do padrão:

$$h(P) = (65 \cdot 256 + 66) \bmod 101 = 16\,706 \bmod 101.$$

$$16\,706 = 165 \cdot 101 + 41 \Rightarrow h(P) = 41.$$

Hash de $T[0..1] = \text{"AB"}$: mesmo cálculo $\Rightarrow h_0 = 41$. **Match!** Confirma byte-a-byte: $\text{"AB"} = \text{"AB"}$
✓.

Roll para $T[1..2] = \text{"BX"}$: ($b^{L-1} = 256$)

$$\begin{aligned} h_1 &= (256 \cdot (h_0 - 65 \cdot 256) + 88) \bmod 101 \\ &= (256 \cdot (41 - 16\,640) + 88) \bmod 101. \end{aligned}$$

Trabalhando em mod 101: $256 \equiv 54$; $65 \cdot 256 \equiv 65 \cdot 54 = 3510 \equiv 76$ (pois $34 \cdot 101 = 3434$).
Então $h_1 \equiv 54 \cdot (41 - 76) + 88 \equiv 54 \cdot (-35) + 88 \equiv -1802 \equiv 16 \pmod{101}$.

Conferência direta: $h(\text{"BX"}) = 66 \cdot 256 + 88 = 16\,984 \equiv 16 \pmod{101}$. ✓

Próximo roll $\rightarrow T[2..3] = \text{"XA"}$, **depois** $T[3..4] = \text{"AB"}$: hash voltará a 41 $\Rightarrow$ segundo match (confirmado byte-a-byte).

- **DJB2** (Daniel J. Bernstein, 1991):

```
unsigned long h = 5381;
while ((c = *s++))
    h = ((h << 5) + h) + c;    // h * 33 + c
```

Simples, rápido, popular em código C clássico.

- **FNV-1a**: multiplicação por primo grande + XOR. Ainda usado em powershell, sqlite, redes.
- **MurmurHash3** (Austin Appleby, 2008): pipeline de *mix* (multiplicação, rotação, XOR). Padrão em Hadoop, Cassandra, Redis.
- **CityHash** / **FarmHash** (Google) e **xxHash**: explora SIMD; > 10 GB/s.
- **SipHash** (Aumasson–Bernstein, 2012): *criptograficamente seguro* e rápido; adotado em Python, Ruby, Rust após os ataques de hash-flooding.

Exemplo numérico: DJB2 sobre "cat"



DJB2: $h = 5381$; para cada char c : $h = 33h + c$. Todas as contas em $\mathbb{Z}_{2^{32}}$ (aqui ignoramos overflow porque o valor cabe em 32 bits).

Passo	h atual	Operação	Novo h
início	5 381	—	5 381
ler c (99)	5 381	$33 \cdot 5\,381 + 99 = 177\,573 + 99$	177 672
ler a (97)	177 672	$33 \cdot 177\,672 + 97 = 5\,863\,176 + 97$	5 863 273
ler t (116)	5 863 273	$33 \cdot 5\,863\,273 + 116$	193 488 125

Verificações: $33 \cdot 5\,381 = 32 \cdot 5\,381 + 5\,381 = 172\,192 + 5\,381 = 177\,573$.

$33 \cdot 177\,672 = 32 \cdot 177\,672 + 177\,672 = 5\,685\,504 + 177\,672 = 5\,863\,176$.

$33 \cdot 5\,863\,273 = 5\,863\,273 \cdot 32 + 5\,863\,273 = 187\,624\,736 + 5\,863\,273 = 193\,488\,009;$
 $+116 = 193\,488\,125$.

Compare com Java: para "cat", Java = 98 262, DJB2 = 193 488 125. Mesma string, "digital" totalmente diferente.

- Mesmo um bom hash de string pode colidir por azar quando aplicamos $\text{mod } p$.
- **Truque competitivo (programação competitiva, bioinformática):** calcule *dois* hashes independentes

$$H(s) = (h_{p_1, b_1}(s), h_{p_2, b_2}(s))$$

com primos p_1, p_2 distintos e bases b_1, b_2 .

- Duas strings só são consideradas iguais se *ambos* os hashes baterem.
- Probabilidade de falso positivo: $\approx 1/(p_1 p_2) \approx 10^{-18}$.
- Útil em: comparação de substrings de DNA, *longest common substring*, deduplicação em larga escala.

Endereçamento Aberto

Definição: endereçamento aberto

Um esquema de resolução de colisões em que **todas as chaves são armazenadas na própria tabela $T[0 \dots m - 1]$** , sem listas ou estruturas auxiliares. Para cada chave k , define-se uma **sequência de sondagem**

$$\langle h(k, 0), h(k, 1), \dots, h(k, m - 1) \rangle,$$

uma *permutação* de $\{0, 1, \dots, m - 1\}$. Para inserir/buscar, percorre-se essa sequência até encontrar o slot da chave (ou um slot vazio).

- Restrição estrutural: $n \leq m$ (não há onde guardar mais chaves que slots).
- Vantagens: sem ponteiros, melhor uso de cache, menor consumo de memória.
- Custo: remoção exige cuidado (marcadores *DELETED*) e desempenho

- Todos os elementos ficam *dentro* da tabela: $n \leq m$.
- Quando $T[h(k)]$ está ocupado, tentamos $T[h_1(k)]$, $T[h_2(k)]$, $\dots$ até achar slot vazio.
- Definimos $h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$ tal que

$$\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$$

é uma *permutação* de $\{0, \dots, m-1\}$.

- Inserção tenta a sequência de *sondagem* até encontrar vazio.

Ideia intuitiva: se o slot $h'(k)$ está ocupado, *olhe o próximo*, depois o seguinte, e assim por diante – como procurar uma vaga andando em linha reta pelo estacionamento.

Sondagem linear

$$h(k, i) = (h'(k) + i) \bmod m, \quad i = 0, 1, \dots, m - 1.$$

- **Prós:** simples de implementar; ótimo para a *cache* (acessos sequenciais).
- **Contra:** sofre de *aglomeração primária* – uma vez formado um cluster, ele cresce ainda mais rápido (chaves que caem em qualquer slot do cluster o estendem).
- Apenas m sequências de sondagem distintas (uma por slot inicial).

Problema da linear: clusters contíguos. **Solução:** *saltos crescentes* em vez de passos unitários, espalhando a sondagem pela tabela.

Sondagem quadrática

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m, \quad c_2 \neq 0.$$

- **Prós:** elimina a aglomeração primária; passos 1, 4, 9, 16, ... pulam clusters.
- **Contra:** *aglomeração secundária* – duas chaves com mesmo $h'(k)$ percorrem a mesma sequência.
- Ainda m sequências distintas; c_1 , c_2 , m precisam ser escolhidos para a sequência cobrir toda a tabela.

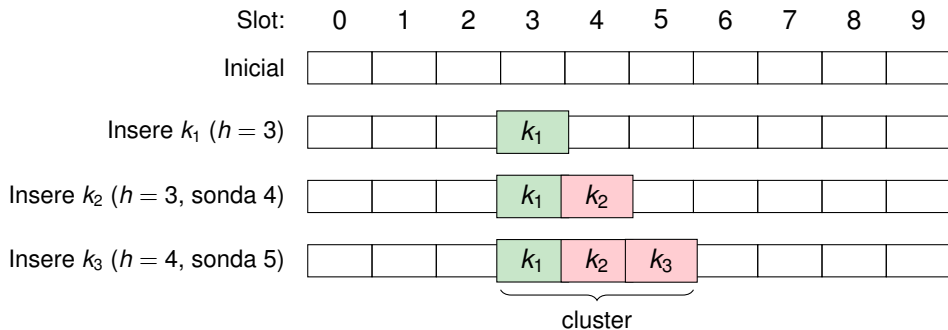
Problema das duas anteriores: o *passo* de sondagem depende só de $h'(k)$.

Ideia: usar uma *segunda função hash* para que o passo dependa da própria chave.

Duplo hashing

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m.$$

- **Prós:** chaves distintas com mesmo h_1 usam passos diferentes $\Rightarrow$ elimina aglomeração secundária.
- Gera $\Theta(m^2)$ sequências de sondagem distintas (aproxima o ideal de *hashing uniforme*).
- **Requisito:** $h_2(k)$ deve ser *coprimo* com m (ex.: m primo, ou $m = 2^p$ com h_2 ímpar) para a sequência ser uma permutação.



Sondagem linear cria *aglomerados*: uma colisão atrai novas colisões.

Tabela $m = 11$, $h(k) = k \bmod 11$. Estado atual:

0	1	2	3	4	5	6	7	8	9	10
22	88	–	14	25	36	47	–	–	9	–

Buscar $k = 36$:

- Sonda 0: $h(36) = 36 \bmod 11 = 3$. $T[3] = 14 \neq 36$. **Continua.**
- Sonda 1: $T[4] = 25 \neq 36$. **Continua.**
- Sonda 2: $T[5] = 36$. **Achou!** Custo: 3 acessos.

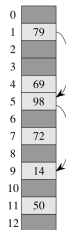
Buscar $k = 100$ (**ausente**):

- $h(100) = 100 \bmod 11 = 1$. Sondas: $T[1] = 88$, $T[2] = \text{vazio} \Rightarrow$ **não está.**

Por que slot vazio termina a busca

Em endereçamento aberto sem remoções, encontrar vazio $\Rightarrow$ chave nunca foi inserida (caso contrário, ela teria parado aqui). Remoções exigem marcador DELETED para preservar a invariante.

- Tabela de tamanho $m = 13$.
- $h_1(k) = k \bmod 13$
- $h_2(k) = 1 + (k \bmod 11)$
- Inserção (nessa ordem):
79, 69, 72, 98, 14, 50.
- Cada chave usa um *passo* diferente, evitando aglomerados.



0	
1	79
2	
3	
4	69
5	98
6	
7	72
8	
9	14
10	
11	50
12	

Estado da tabela após as inserções.

$$h_1(k) = k \bmod 13, h_2(k) = 1 + (k \bmod 11), h(k, i) = (h_1 + i \cdot h_2) \bmod 13.$$

k	h_1	h_2	Sequência de sondagem	Slot final
79	1	$1 + 2 = 3$	1	1
69	4	$1 + 3 = 4$	4	4
72	7	$1 + 6 = 7$	7	7
98	7	$1 + 10 = 11$	7 (ocup. por 72), $(7 + 11) \bmod 13 = 5$	5
14	1	$1 + 3 = 4$	1 (ocup.), $(1 + 4) = 5$ (ocup.), $(1 + 8) = 9$	9
50	11	$1 + 6 = 7$	11	11

Verificações: $79 \bmod 13 = 79 - 6 \cdot 13 = 79 - 78 = 1$. $98 \bmod 13 = 98 - 7 \cdot 13 = 98 - 91 = 7$.
 $98 \bmod 11 = 98 - 8 \cdot 11 = 98 - 88 = 10$. $50 \bmod 13 = 50 - 3 \cdot 13 = 50 - 39 = 11$.

Estado final coincide com a Fig. 11.5: slots 1, 4, 5, 7, 9, 11 ocupados.

Hipótese de hashing uniforme: cada chave tem como sequência de sondagem qualquer permutação de $\{0, \dots, m-1\}$ com igual probabilidade.

Resultados (fator de carga $\alpha = n/m < 1$)

- Busca *sem sucesso*: em média $\leq \frac{1}{1-\alpha}$ sondagens.
- Busca *com sucesso*: em média $\leq \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$ sondagens.

α	Busca sem sucesso	Busca com sucesso
0,5	$\approx 2,0$	$\approx 1,39$
0,9	$\approx 10,0$	$\approx 2,56$
0,99	≈ 100	$\approx 4,65$

Mensagem: mantenha α longe de 1 (tipicamente $\leq 0,75$).

Hashing Perfeito

Definição: hash perfeito

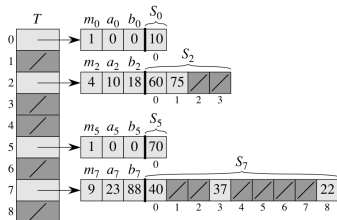
Dado um conjunto **estático** de chaves $K = \{k_1, k_2, \dots, k_n\}$, uma função hash $h: U \rightarrow \{0, 1, \dots, m-1\}$ é dita *perfeita* para K se ela é **injetiva** sobre K – isto é, para todo $k_i, k_j \in K$ com $k_i \neq k_j$, vale $h(k_i) \neq h(k_j)$.

- Sem colisões em $K \Rightarrow$ busca em $O(1)$ **no pior caso**.
- Se, além disso, $m = n$, h é dita **mínima** (sem slots vazios).
- Construção é viável porque K é conhecido em tempo de projeto.

- Aplicável quando o conjunto de chaves K é **estático** (conhecido em tempo de projeto).
- Exemplos: palavras-chave de uma linguagem de programação, símbolos de um arquivo de objeto.
- Objetivo: garantir que a busca custe $O(1)$ *no pior caso*.

Estratégia em dois níveis

1. Hash primário h distribui as n chaves em $m = n$ slots.
2. Cada slot j com n_j chaves tem uma sub-tabela S_j de tamanho $m_j = n_j^2$, com função h_j escolhida para não ter colisões.



Hashing perfeito armazenando $K = \{10, 22, 37, 40, 60, 70, 75\}$.

- **Tabela primária T :** hash $h(k) = ((ak + b) \bmod p) \bmod m$ distribui as $n = 7$ chaves em $m = 9$ slots; cada slot j guarda os parâmetros (m_j, a_j, b_j) da sub-tabela.
- **Sub-tabelas S_j :** tamanho $m_j = n_j^2$ (ex.: slot 2 tem $n_2 = 2$ chaves $\Rightarrow m_2 = 4$); a função h_j é escolhida até não haver colisão.
- Slots vazios ($n_j = 0$) não alocam sub-tabela; a busca por k faz *dois* acessos: $T[h(k)]$ e $S_{h(k)}[h_{h(k)}(k)]$.

Por que $m_j = n_j^2$ funciona?

Teorema 11.9: se armazenamos n chaves em uma tabela de tamanho $m = n^2$ com h escolhida *aleatoriamente* de uma família universal, a probabilidade de haver alguma colisão é $< 1/2$.

Consequência: basta tentar algumas funções h_j até encontrar uma sem colisões em cada sub-tabela.

Espaço total

$$E \left[\sum_{j=0}^{m-1} n_j^2 \right] < 2n.$$

Ou seja, em média gastamos $O(n)$ de espaço para garantir $O(1)$ de busca *no pior caso*.

Síntese

Cenário	Estrutura recomendada	Porquê
Universe pequeno e denso	Endereçamento direto	$O(1)$ trivial, sem hash.
Conjunto dinâmico, α pode crescer	Encadeamento	Tolera $\alpha > 1$.
Memória limitada, $\alpha < 0,75$	Endereçamento aberto (duplo hashing)	Sem ponteiros, cache-friendly.
Conjunto estático, busca crítica	Hashing perfeito	$O(1)$ no pior caso.
Adversário malicioso	Família universal de hashes	Independência das chaves.

- Tabelas hash transformam $\Theta(\log n)$ ou $\Theta(n)$ em $\Theta(1)$ *esperado*.
- Função hash boa = rápida + bem distribuída.
- **Sempre há colisões** – a engenharia está em como tratá-las.
- Fator de carga $\alpha = n/m$ é o parâmetro mestre da análise.
- Para garantias contra adversários: *universal hashing*.
- Para $O(1)$ no pior caso (chaves estáticas): *hashing perfeito*.

1. Aglomeração

Insira as chaves 10, 22, 31, 4, 15, 28, 17, 88, 59 em uma tabela de tamanho $m = 11$ usando:

1. encadeamento com $h(k) = k \bmod 11$;
2. sondagem linear com o mesmo h ;
3. duplo hashing com $h_2(k) = 1 + (k \bmod 10)$.

Compare os números de sondagens.

2. Fator de carga

Mostre que, sob hashing uniforme simples e encadeamento, a busca sem sucesso custa $\Theta(1 + \alpha)$ em média.

- Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. *Introduction to Algorithms*. 2.ed. MIT Press, 2001. **Capítulo 11 – Hash Tables.**
- Knuth, D. E. *The Art of Computer Programming, Vol. 3: Sorting and Searching*. 2.ed. Addison-Wesley, 1998. **Seções 6.4.**
- Carter, J. L.; Wegman, M. N. “Universal classes of hash functions.” *Journal of Computer and System Sciences*, 18(2):143–154, 1979.

Figuras 11.1–11.6 extraídas da edição 2001 do Cormen *et al.* para fins educacionais.